IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts, Science, Commerce, and Management

Project – Low Level Design

on

Design and Development of Natural Language Interface for E-commerce Infrastructure

Student Name: Alena Latheesh

Industry Mentor: **Guided by: Mr. Praveen**

Table of Contents

1. Introduction 3
1.1. Scope of the document 3
1.2. Intended Audience 3
1.3. System overview 3
2. System Design 4
2.1. Application Design 4
2.2. Process Flow 7
2.3. Information Flow ..... **Error! Bookmark not defined.**
2.4. Components Design 88
2.5. Key Design Considerations ..... 10**Error! Bookmark not defined.**
2.6. API Catalogue 12
3. Data Design 13
3.1. Data Model 13
3.2. Data Access Mechanism 14
3.3. Data Retention Policies 14
3.4. Data Migration 14
4. Interfaces 15
5. State and Session Management 17
6. Non-Functional Requirements 19
6.1. Security Aspects 19
6.2. Performance Aspects 19
7. References 21

1. Introduction

This document provides a detailed Low-Level Design (LLD) focusing on internal implementation details, including function-level behaviour, DOM manipulation, API handling, and data validation across the hybrid Web/CLI application.

1.1. Scope of the Document

This document includes:

- Web module-level design (static/app.js, index.html)
- CLI module-level design (main.py)
- Backend API routing and NLP logic (app.py)
- JSON data storage mechanisms

1.2. Intended Audience

- Developers
- Project evaluators
- Academic reviewers

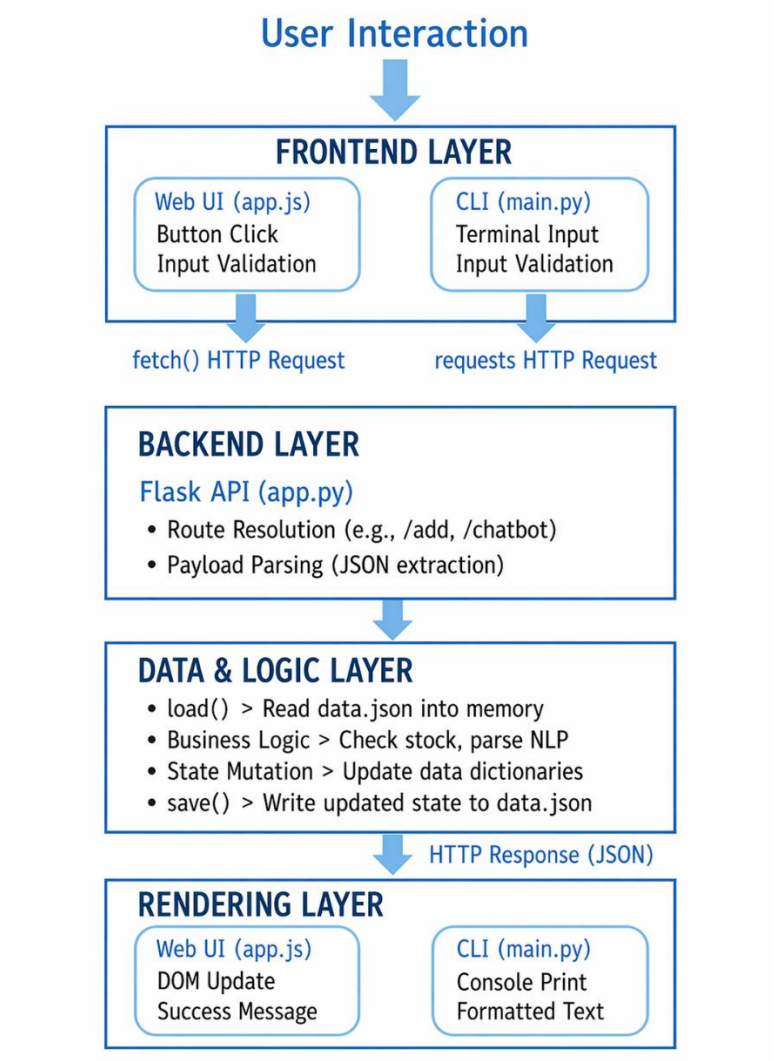
1.3. System Overview

The system comprises a primary Web UI and an optional CLI that interact with a centralized Flask backend.

- static/app.js: Handles browser interactions and dynamic DOM rendering.
- main.py: Handles terminal interactions.
- app.py: Processes REST routes, chatbot queries, and file I/O.
- data.json: The flat-file data store.

2. System Design

Execution Flow:



2.1. Application Design

- WEB UI MODULE (static/app.js)

This file is the "brain" of your browser experience. It operates entirely on the user's computer (client-side) and has three primary jobs:

- **Capturing User Events:** It actively listens for things the user does, such as clicking an "Add to Cart" button, hitting "Enter" in the chat input, or clicking the checkout button.
- **Asynchronous Updates (Fetch & DOM Manipulation):** When an event happens, it uses the JavaScript fetch() API to send a background request to the server. When the server replies, app.js uses **DOM manipulation** to update specific parts of the screen (like changing the cart counter from "0" to "1") *without* forcing the user to refresh the entire web page.
- **Chatbot Drawer State:** It controls the visual toggling (opening and closing) of the side-panel chatbot and handles taking the text the user types and sending it off to the backend for processing.

➢ CLI MODULE (main.py)

This is your terminal-based alternative to the website. It runs as a continuous loop in the console and acts as a strict translator between the user and the API.

- **api_get() and api_post():** Instead of the browser's fetch() API, this Python script uses the requests library to talk to your server. These specific functions are wrappers that handle network errors gracefully (e.g., if the server is offline, it prevents the app from completely crashing).
- **Formatted Text Tables:** Because a terminal cannot render HTML or CSS buttons, this module takes the raw JSON data sent back from the server and uses Python string formatting to print neat, aligned text-based menus and product lists for the user to read.

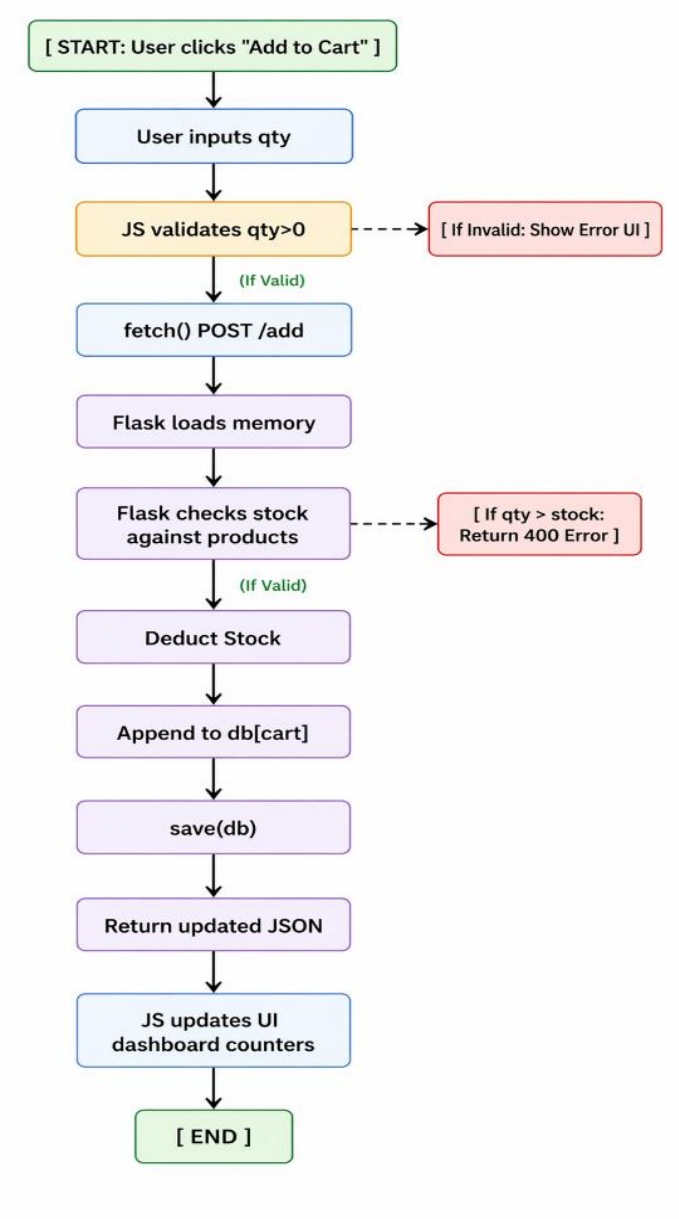
➢ BACKEND MODULE (app.py)

This is the core engine of your project. It is the Flask web server that securely processes all the logic, regardless of whether the request came from the Web UI or the CLI.

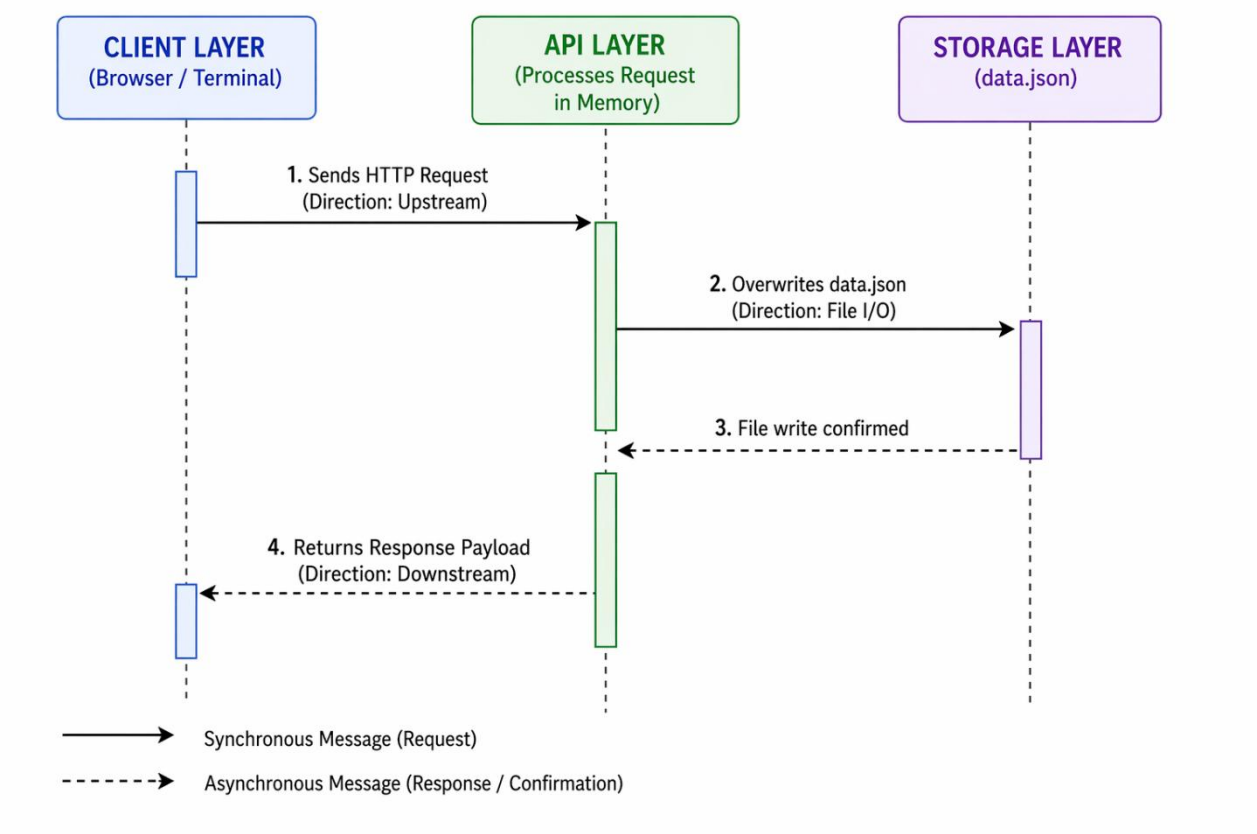
- **load() / save() (File Operations):** Because you are using a flat-file database, these functions are your direct link to data.json. load() reads the JSON file and converts it into a workable Python dictionary. Once the backend changes the data (like subtracting 1 from a product's stock), save() immediately overwrites the file on the hard drive to ensure the data is permanently updated.
- **build_chat_response() (NLP Parser):** This function acts as the "brain" of your chatbot. When a user sends a message, this function scans the text string for specific keyword triggers (like "cheap", "delivery", or "refund"). If it finds a match, it constructs a highly structured JSON response (containing a title, a message, and a list of bullet points) and sends it back to the frontend to be displayed as a chat bubble.

2.2. Process Flow

Add to car Flow:



2.3. Information Flow Sequence Diagram



2.4. Components Design

- Frontend Components

This is the interactive layer—the part of the system that actually "talks" to the user. Because your project is a hybrid, this component handles two completely different environments:

- **Event Listeners (app.js):** In your web dashboard, JavaScript code actively "listens" for user interactions. When a user clicks an "Add to Cart" button or hits "Enter" in the chatbot, the event listener catches that exact action and immediately triggers a function (like firing off a fetch() request).

- **DOM Manipulators (app.js):** After the server sends back a successful response, JavaScript modifies the Document Object Model (DOM). Instead of refreshing the entire web page, it dynamically changes specific HTML elements in real-time—like swapping an "Out of Stock" badge or increasing the cart total counter.
- **Terminal Prompt Loops (main.py):** For the CLI version, you can't rely on clicks. Instead, you use Python while True loops. These loops keep the application running continuously in the terminal. They print a menu, pause to wait for the user to type a command, send that command to the API, print the response, and loop right back to the start.

➢ API Component

This is the gatekeeper located inside your Flask backend (app.py). Its entire job is to intercept incoming network traffic, unpack it, and make sure it is safe before passing it onto the main system.

- **Request JSON Parsing (request.get_json()):** When your frontend sends data (like the product name and quantity), it travels over the network as a raw text string. Flask uses this method to catch that string and parse it into a standard Python dictionary so your code can easily read things like data["qty"].
- **Exception Handling (try-except):** This is your server's armor. If a user tries to break the system by typing "ten" instead of the number 10, Python will throw a ValueError when it tries to do math with a word. A try-except block catches that error safely and returns a clean HTTP 400 "Bad Request" message back to the user, preventing the entire Flask server from crashing.

➢ Business Logic

This is the core e-commerce "brain" of the project. It doesn't know anything about HTML files or network requests; it only executes the rules of your store.

- **Stock Deduction Algorithms:** When an order is placed, the logic steps in to verify that requested_qty is less than or equal to available_stock. If the math checks out, it automatically deducts the units from the inventory dictionary and appends them to the cart dictionary.
- **Price Aggregation:** The system loops through everything currently sitting in the user's cart, looks up the base price for each item in the product catalog, multiplies it by the selected quantity, and calculates the final checkout total.

➢ Data Component

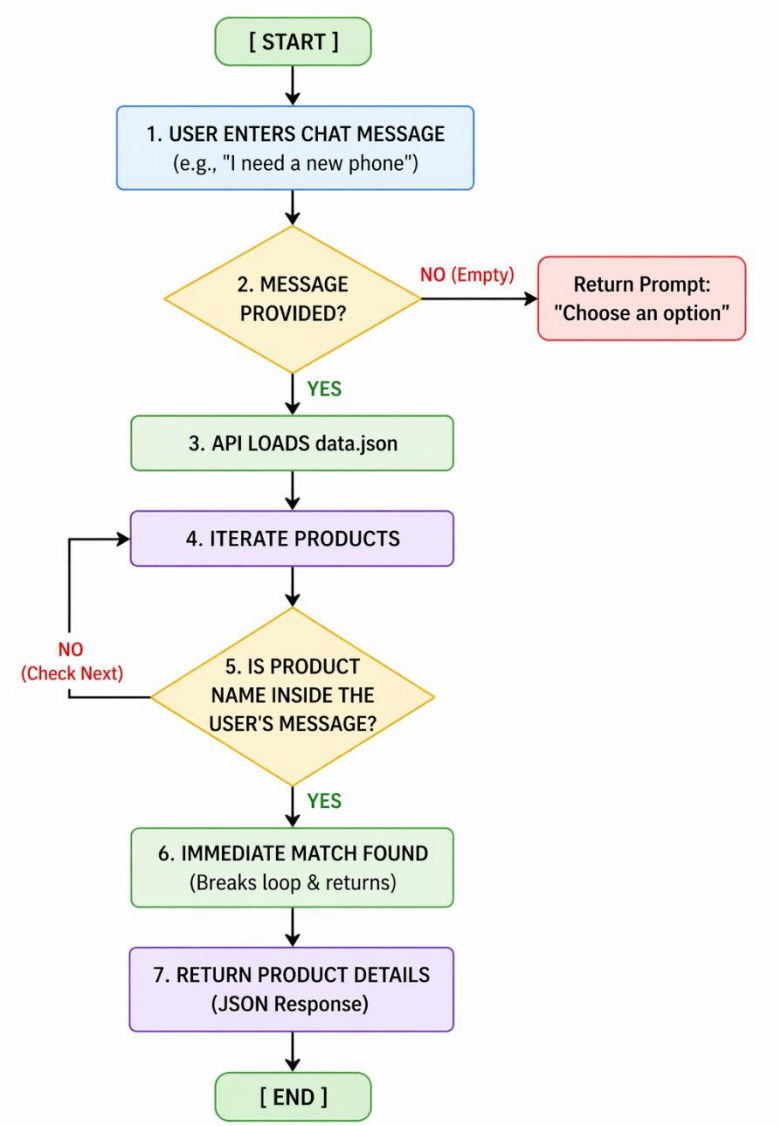
This layer is entirely dedicated to persistence. Since you bypassed traditional databases (like MySQL or MongoDB) to keep the architecture lightweight, this component manages all interactions with your hard drive.

- **open():** The native Python command that safely opens the data stream to your local data.json file.
- **json.load():** Reads the raw text file from the disk and translates (deserializes) it into Python memory. This turns your flat file into active, searchable lists and dictionaries.
- **json.dump():** The crucial final step. After the Business Logic layer finishes making changes, this function takes the active Python dictionaries and translates (serializes) them back into standard JSON text, overwriting data.json so the cart and inventory changes are saved permanently.

2.5. Key Design Considerations

Dual-layer input validation: Quantity types and limits are checked client-side (JS/CLI) to reduce server load, and strictly enforced again server-side (Flask) to guarantee data consistency.

Simplified product search flowchart (substring matching):



2.6. API Catalogue

API	Method	Input	Output	Internal Processing
/products	GET	None	List	load()
/cart	GET	None	List	load()
/add	POST	product, qty	msg	validate → update → save
/checkout	GET	None	msg	check cart → clear → save

3.Data Design

File: data.json

Stores:

- Product list
- Cart items

3.1. Data Model

Field Name	Data Type	Description	Example
name	String	Unique identifier for the product	"Laptop"
price	Integer	Cost per unit	50000
stock	Integer	Quantity available in inventory	10
qty	Integer	Number of items added to cart	2

3.2. Data Access Mechanism

- File read using open()
- JSON parsed using json.load()
- Data updated in memory
- Saved using json.dump()

3.3. Data Retention Policies

File is persistently modified. Cart array is emptied (db["cart"] = []) inside the /checkout route.

3.4. Data Migration

- Not implemented

4. Interfaces

- Browser to Backend (app.js → app.py)

This interface handles the communication between the user's web browser and your Flask server.

- **The Technology:** It uses the native JavaScript fetch() API to send asynchronous HTTP requests (GET and POST) over the network.
- **Asynchronous Promises:** Because fetch() is asynchronous, it returns a "Promise." This means the browser sends the request and continues running other code (like keeping animations smooth or allowing the user to keep scrolling) while waiting for the server to reply.
- **Resolving to .json():** When the Flask server sends an HTTP response back, it arrives as a raw text stream. The app.js file chains the .json() method to the response Promise. This forces the browser to parse that raw text into a structured, readable JavaScript Object, which the frontend then uses to update the DOM (like rendering the new cart total).

- CLI to Backend (main.py → app.py)

This interface handles the communication between your terminal window and the exact same Flask server.

- **The Technology:** It relies on the external Python requests library to construct and send synchronous HTTP requests.
- **Synchronous Execution:** Unlike the browser, the CLI interface operates synchronously. When requests.get() or requests.post() is called, the terminal pauses and waits for the server to process the request and return the JSON data before it prints the next menu prompt.

- **The 5-Second Timeout:** This is a critical resilience feature. Networks are unpredictable, and sometimes the Flask server might be offline or rebooting. By enforcing timeout=5, you ensure that if the server doesn't respond within exactly 5 seconds, the requests library will cleanly abort the attempt. Your try-except block then catches this timeout and prints a friendly [ERROR] Server not reachable message, preventing the entire CLI program from freezing or crashing.

➢ Backend to Data (app.py → data.json)

This interface manages how your application state is permanently saved to the hard drive.

- **The Technology:** It uses Python's built-in open() function combined with the native json library (json.load() and json.dump()).
- **File Streams:** Because you are bypassing a traditional database, the backend creates a direct Input/Output (I/O) stream to the data.json file. json.load() reads the file stream from the disk and deserializes it into Python dictionaries in active memory. After the business logic modifies those dictionaries, json.dump() serializes them back into text and streams them to the disk, completely overwriting the old file.
- **UTF-8 Encoding:** Specifying encoding="utf-8" in the open() function is a crucial safety measure. It ensures that the system uses the universal UTF-8 standard for translating text into binary. This guarantees that any special characters—such as the Indian Rupee symbol (Rs.), accented letters, or even emojis—are perfectly preserved during the read/write cycle without triggering a UnicodeDecodeError that would crash your server.

5.State and Session Management

State Management

The architectural paradigm of this project relies entirely on a **stateless API design**. This means the Flask server operates with complete "amnesia" between user interactions.

- **Complete Independence:** Every HTTP request from the Web UI or CLI must contain all the necessary context to execute the action. The server does not store any transient variables (like a running cart total) in its active memory between clicks.
- **Persistent Delegation:** The application state—most notably the contents of the user's shopping cart—is delegated entirely to the persistent data layer (data.json).
- **Aggregation Loops:** Because the server forgets the state after every response, dynamic data must be recalculated on the fly. Functions like build_cart_summary read the latest values directly from the data.json file and loop through the data to calculate real-time totals on every single request.
- **Fault Tolerance:** This design ensures that the cart persists between requests and even if the server program is completely restarted or crashes, because the absolute source of truth lives safely on the hard drive.

Session Management

The system is explicitly designed as a localized demonstration environment, which heavily dictates its approach to user sessions.

- **No Active Tracking:** There is **no session management** currently implemented. The backend does not utilize Flask session cookies, JSON Web Tokens (JWTs), or any authorization scopes.

- **Single-User Simulation:** The immediate implication of this design is that the application operates strictly as a single-tenant environment. Because the server cannot distinguish between different users, everyone accessing the API is viewing and modifying the exact same, shared shopping cart.
- **Design Rationale:** Concurrent session management, user authentication protocols, and login architectures were intentionally bypassed by design. This omission drastically reduces system complexity and keeps the architectural focus entirely on executing the core e-commerce workflow (adding to cart, checking stock, and processing checkouts).

6. Non-Functional Requirements

6.1. Security Aspects

- **Crash Prevention via Exception Handling:** * When the frontend sends a quantity payload, the Flask route must convert the text string to an integer using int().
    - If a user sends invalid data (e.g., "two", "-5", or an empty string), it triggers a ValueError or TypeError.
    - try-except blocks safely catch these exceptions and return an HTTP 400 "Bad Request" message, preventing the entire Flask server from crashing .
- **Graceful JSON Fallbacks:** * Routes like /add or /chatbot expect a JSON payload. If a request is missing a body or has broken formatting, Flask normally aborts and throws an error.
    - Using request.get_json(silent=True) suppresses the crash and returns None.
    - The code then uses or {} to convert None into an empty dictionary, guaranteeing the backend always has a safe object to evaluate and neutralizing malformed requests.

6.2. Performance Aspects

- **Network Optimization (/dashboard-data):**
    - The Web UI requires three datasets to render: the product catalog, cart contents, and overview totals.
    - Making three separate fetch() calls would cause network latency and force the server to read data.json multiple times.
    - The /dashboard-data route batches all initial rendering data into a single HTTP request, drastically reducing load times and making the UI feel instantaneous.

- **Processing Efficiency:**
    - Due to the stateless design, the backend recalculates totals from scratch on every request.
    - Instead of standard, slower for loops, the code uses Python's built-in dictionary and list comprehensions (e.g., mapping product prices).
    - Comprehensions execute at the C-language level, making them highly optimized, memory-efficient, and significantly faster, thereby minimizing processing overhead.

7.References

- **MDN Web Docs (Fetch API, DOM Manipulation)**

- **Fetch API**: Used as the primary technical reference for implementing the asynchronous fetch() calls in static/app.js. It guided the structure of the JavaScript Promises used to send POST requests to the /add and /chatbot endpoints and GET requests to /dashboard-data.
- **DOM Manipulation**: Provided the standard methodologies for selecting and updating HTML elements in index.html. This ensured that the dashboard UI could re-render cart totals and product lists dynamically without requiring a full page refresh.

- **Flask Official Documentation**

- **Routing and Requests**: Utilized to design the RESTful API architecture in app.py. It served as the guide for using decorators like @app.route() to map endpoints and request.get_json() to handle incoming JSON payloads.
- **Template Rendering**: Guided the use of render_template() to inject backend data—such as the product list and user profile—directly into the initial index.html view.

- **Python json and requests Documentation**

- **json Library**: Served as the core reference for the Data Component. It provided the logic for json.load() to deserialize the database into Python dictionaries and json.dump() to serialize memory states back to the data.json file.
- **requests Library**: Used for the CLI-to-Server interface in main.py. It guided the implementation of synchronous GET and POST calls, including the use of 5-second timeout protocols to ensure client-side resilience against server unreachability.